CRIME REPORT MANAGEMENT SYSTEM

21CSC205P Database Management Systems

A MINI PROJECT REPORT

Submitted by

Shantanu Ghandhare [RA2411026010488]

Aaryan Dipnaik [RA2411026010479]

Under the Guidance of

Dr. Sasi Rekha Sankar

(Assistant Professor, Department of Computer Intelligence)

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in (AI & ML)



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

Certified that the Project report titled "**Crime Report Management System**" is the bonafide work of **Shantanu Ghandhare [RA2411026010488] and Aaryan Dipnaik [RA2411026010479]** who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

SIGNATURE

Dr. Sasi Rekha Sankar

ASSISTANT PROFESSOR
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE

SIGNATURE

DR. R. ANNIE UTHRA

PROFESSOR & HEAD
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE

ABSTRACT

The Crime Report Management System is a database-driven application designed to efficiently store, manage, and retrieve criminal records within a law enforcement environment. The system aims to digitalize and streamline the management of crime-related information such as prisoner details, crime reports, First Information Reports (FIR), investigation records, evidence details, court case information, and user roles including administrators, police officers, and court officials. By organizing this data in a structured and centralized database, the system significantly improves accuracy, accessibility, and reliability when compared to traditional manual record-keeping methods, which are often prone to redundancy, inconsistency, and data loss.

The primary objective of this project is to apply fundamental Database Management System (DBMS) concepts to design a robust, scalable, and efficient data management solution. The system is developed using Entity-Relationship (ER) modeling to clearly identify key entities such as Criminal, Police Officer, Police Station, FIR, Case File, Court, Evidence, and User. The relationships between these entities are carefully defined to reflect real-world interactions, such as FIRs being registered by officers, criminals being linked to FIRs, cases being handled by courts, and evidence being associated with specific cases.

Normalization techniques up to higher normal forms are applied to eliminate redundancy, prevent anomalies, and ensure data integrity. The database design ensures that each piece of information is stored logically and efficiently, enabling consistent updates and minimizing duplication. Structured Query Language (SQL) is used extensively to implement database operations including table creation, data insertion, updating records, deletion, and retrieval of information. Advanced SQL queries such as joins, subqueries, aggregate functions, and views are used to extract meaningful insights from the stored data.

The system supports various real-world operations such as registering FIRs, maintaining detailed criminal profiles, assigning cases to courts, managing evidence records, and tracking the progress of investigations and hearings. It also incorporates important DBMS concepts such as constraints, triggers, and procedures to enforce business rules and automate tasks. Furthermore, transaction management is implemented to ensure Atomicity, Consistency, Isolation, and Durability (ACID properties), while concurrency control mechanisms handle multiple users accessing and modifying the database simultaneously without conflicts.

The project also demonstrates recovery mechanisms to maintain database stability in case of system failures, ensuring that no critical data is lost. Overall, the Crime Report Management System provides a secure, organized, and efficient approach to managing criminal records. It serves as a practical implementation of core DBMS principles in a real-world scenario and highlights the importance of database systems in modern law enforcement and administrative processes.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	**Problem Understanding, Identification of Entity and Relationships, Construction of DB using ER Model**	9-12
	1.1 Introduction	
	1.2 Motivation	
	1.3 Scope	
	1.4 Problem Statement	
	1.5 Project Requirements	
	1.6 Identification of Entity and Relationships	
	1.7 Construction of DB using ER Model	
2.	**Design of Relational Schemas, Creation of Database and Tables**	13-21
	2.1 Relational Schema	
	2.2 Description Table	
	2.3 Creation of Database and Tables - DDL Command	
	2.4 Insertion of tuples into the table – DML commands	

3.	**Complex queries based on the concepts of Constraints, Sets, Joins, Views, Triggers and Cursors.**	**22-25**
	3.1 Adding Constraints and queries based on constraints	
	3.2 Queries based on Aggregate Functions	
	3.3 Complex queries based on Sets	
	3.4 Complex queries based on Subqueries	
	3.5 Complex queries based on Joins	
	3.6 Complex queries based on views	
	3.7 Complex queries based on Triggers	
	3.8 Complex queries based on Cursors	
4.	**Analyzing the Pitfalls, identifying the Dependencies, and applying Normalization**	**26-33**
	4.1 Analyse the Pitfalls in Relation	
	4.2 First Normal Form	
	4.3 Second Normal Form	
	4.4 Third Normal Form	

4.5 BCNF

4.6 Fourth Normal Form

4.7 Fifth Normal Form

5. Implementation of Concurrency Control and Recovery Mechanisms **34-46**

5.1 Introduction to Transactions

5.2 Transaction Control Language (TCL)

5.3 Execution of Transactions and Results

5.4 Concurrency Control

6. FRONT-END AND BACK-END CODE OF CRIME REPORT MANAGEMENT SYSTEM **47-54**

6.1 Front-End Module Codes

6.2 Database Connectivity

7. Result and Discussions **55-64**

7.1 Screenshots of Front-End

7.2 Screenshots of Backend and Database

LIST OF FIGURES

CHAPTER NO.	TITLE	PAGE NO.
1.1	ER Diagram	12
2.1	Relational Schema	13

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.	User	15
2.	Admin	15
3.	Police_Station	16
4.	Police_officer	16
5.	FIR	17
6.	Criminal	17
7.	FIR_Criminal	18
8.	Criminal_Record	18
9.	Court	19
10.	Case_File	19
11.	Evidence	20
12.	Lock Table	43

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE CRIME REPORT MANAGEMENT SYSTEM

1.1 Introduction

In modern law enforcement systems, managing crime-related information efficiently is crucial for maintaining public safety and ensuring justice. Traditional manual methods of storing criminal records, FIR details, investigation data, and court proceedings often lead to data redundancy, inconsistency, and difficulty in retrieving accurate information.

The Crime Report Management System is designed using Database Management System (DBMS) concepts to store, organize, and manage crime-related data in a structured and centralized manner. This project models real-world processes such as crime reporting, investigation, criminal tracking, and court case handling using Entity-Relationship (ER) modelling and converts them into a well-defined relational database. The system ensures data integrity, minimizes redundancy, and enables efficient data retrieval through SQL queries.

1.2 Motivation

The motivation behind developing this project is to understand and apply core DBMS concepts to a real-world domain such as law enforcement. Crime management involves handling large volumes of interconnected data including FIR records, criminal profiles, police officer details, evidence records, and court cases, which cannot be efficiently managed using traditional methods.

By implementing this system using DBMS principles, the project demonstrates the practical application of:
- Entity-Relationship modelling
- Primary and foreign keys
- Normalization
- Constraints and relationships
- SQL operations

This project provides hands-on experience in designing a database system that is scalable, reliable, and capable of handling complex relationships.

1.3 Scope

The scope of this project includes the design and implementation of a database system that manages:
• Criminal details and criminal records
• FIR (First Information Report) data
• Police officers and police station information
• Case management and investigation details
• Court information and case proceedings
• Evidence collection and storage

The system focuses only on database design and operations and does not include front-end or application-level implementation. The database is structured to support future enhancements such as analytics, reporting, and integration with external systems.

1.4 Problem Statement

To design and implement a centralized database system using DBMS concepts that can efficiently manage crime reports, criminal records, FIR data, investigation details, and court proceedings while ensuring data integrity, minimizing redundancy, and enabling accurate and efficient data retrieval through structured queries.

1.5 Project Requirements

1.5.1 Functional Requirements

• Store criminal details and their records
• Maintain FIR details including date, time, and description
• Record police officer and police station information
• Associate multiple criminals with a single FIR
• Manage case details and link them to courts
• Store and manage evidence related to cases
• Track case status and progress

1.5.2 Non-Functional Requirements

• Ensure data consistency and integrity using constraints
• Reduce redundancy through normalization
• Support efficient query processing
• Maintain security and controlled access using relational constraints

1.6 Identification of Entity and Relationships

1.6.1 Identified Entities

The following entities are identified in the system:
• USER – stores login and general user information
• ADMIN – manages the system
• POLICE_OFFICER – stores officer details
• POLICE_STATION – stores station details
• FIR – stores crime report information
• CRIMINAL – stores criminal details
• CRIMINAL_RECORD – stores past records of criminals
• CASE_FILE – stores case details
• COURT – stores court information
• EVIDENCE – stores evidence details

1.6.2 Relationships Identified

• A police station has multiple police officers (One-to-Many)
• A police officer registers multiple FIRs (One-to-Many)
• A FIR can involve multiple criminals (Many-to-Many)
• A criminal can be involved in multiple FIRs (Many-to-Many)
• A FIR leads to one case (One-to-One)
• A court handles multiple cases (One-to-Many)
• A case contains multiple evidence records (One-to-Many)
• A criminal has one criminal record (One-to-One)

1.6.3 Generalization and Specialization

The USER entity is specialized into:
• ADMIN
• POLICE_OFFICER

This ISA hierarchy allows shared attributes such as name, email, and contact details to be maintained in the USER entity while role-specific attributes are handled separately.

1.7 Construction of Database Using ER Model for the Crime Report Management System

Based on the identified entities and relationships, an Entity-Relationship (ER) model is constructed to represent the logical structure of the system. The ER diagram includes:

- Strong entities represented using rectangles
  - Weak entities (if any) represented using double rectangles

- Relationships represented using diamonds
  - Attributes represented using ovals
  - Multivalued and derived attributes where necessary
  - Cardinality and participation constraints
  - Generalization and specialization using ISA hierarchy

The ER model serves as a blueprint for converting the conceptual design into a relational schema. Each entity and relationship is mapped into tables using standard DBMS conversion techniques, ensuring referential integrity and normalization.

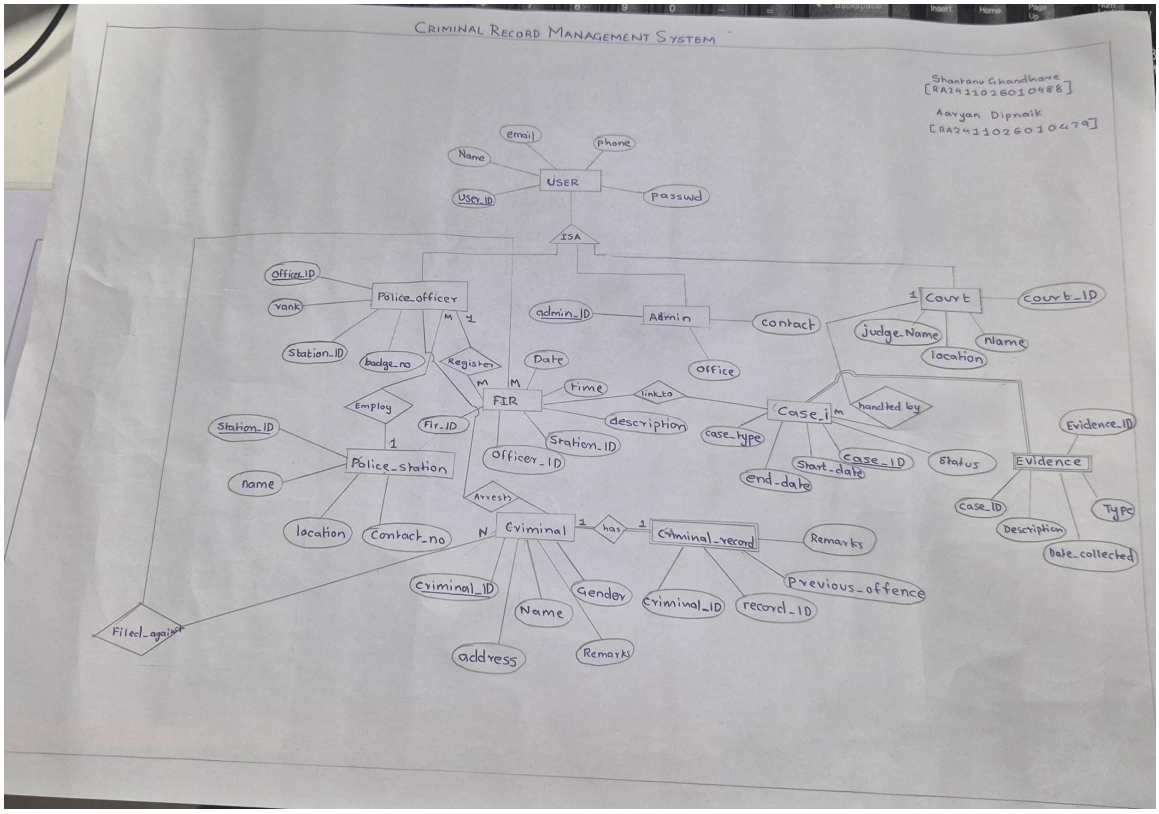


Figure 1.1: ER Diagram of Criminal Record Management System

The ER diagram of the Criminal Record Management System represents the structure of the database by identifying key entities such as User, Admin, Police Officer, Police Station, FIR, Criminal, Case File, Court, and Evidence along with their relationships. The User entity is specialized into Admin and Police Officer, while Police Officers are associated with Police Stations. FIR acts as a central entity linking officers, stations, and criminals, with a many-to-many relationship between FIR and Criminal resolved using the FIR_Criminal table. Criminals have associated criminal records, and each FIR can lead to a Case File handled by a Court. Evidence is linked to cases to store investigation details. Overall, the ER diagram ensures proper organization of data, reduces redundancy, and maintains relationships for efficient management of criminal records.

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for Employee Skill, Appraisal, Feedback & Performance Tracking System

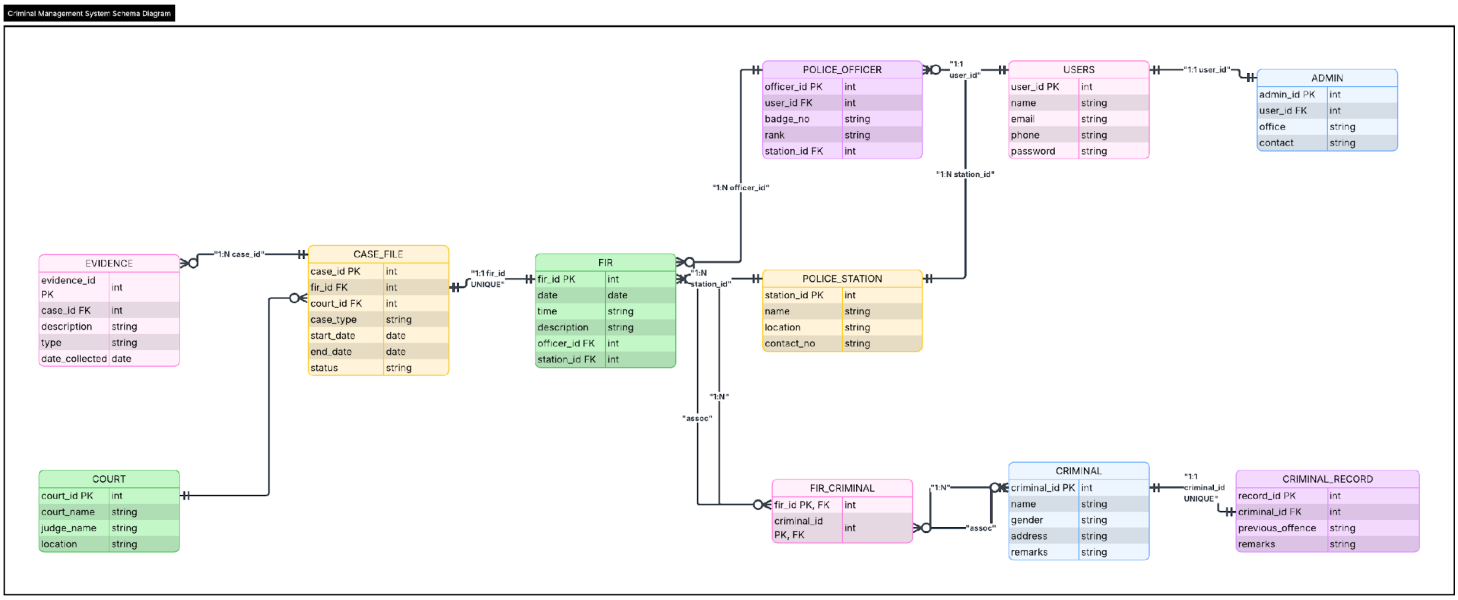


Figure 2.2: Relational Schema for Crime Report Management System

2.1 Relational Schema for Crime Report Management System

The Figure 2.1 depicts the relational schema for the Crime Report Management System. It illustrates the database tables derived from the ER/EER model along with their attributes, primary keys, and foreign key relationships. The schema shows how entities such as Users, Admin, Police_Officer, Police_Station, FIR, Criminal, Criminal_Record, Case_File, Court, and Evidence are structured in relational form.

Junction tables such as **FIR_Criminal** are used to resolve many-to-many relationships between FIR and Criminal entities. Weak or dependent entities are represented with appropriate foreign key constraints, and specialization is handled through separate subtype tables such as Admin and Police_Officer derived from the Users entity.

This relational schema ensures data integrity, minimizes redundancy through normalization, and supports efficient data retrieval using structured SQL queries. It provides a clear representation of how crime data flows from FIR registration to case handling and court proceedings within the system.

Cardinalities in Diagram

User → Admin : **1 : 1** (one user can be one admin)
User → Police Officer : **1 : 1** (one user can be one officer)

Police Station → Police Officer : **1 : N** (one station has many officers)
Police Officer → Police Station : **N : 1**

Police Officer → FIR : **1 : N** (one officer can register many FIRs)
FIR → Police Officer : **N : 1**

Police Station → FIR : **1 : N** (one station can have many FIRs)
FIR → Police Station : **N : 1**

FIR → Criminal : **M : N** (one FIR can involve multiple criminals and one criminal can be in multiple FIRs)
(resolved using FIR_Criminal)

Criminal → Criminal_Record : **1 : N** (one criminal can have multiple records)

FIR → Case_File : **1 : 1** (each FIR leads to one case file) *(can also be 1:N depending on design, but yours is 1:1)*

Court → Case_File : **1 : N** (one court handles many cases)
Case_File → Court : **N : 1**

Case_File → Evidence : **1 : N** (one case can have multiple evidence items)
Evidence → Case_File : **N : 1**

2.2 Description of Tables

Table 2.2.1: Users

• Purpose: Stores general user information such as login credentials and contact details.
• Primary Key: user_id (INT)
• Data Types and Constraints:

1. user_id: INT, Primary Key, Not Null
2. name: VARCHAR(100), Not Null
3. email: VARCHAR(100), Unique, Not Null
4. phone: VARCHAR(15), Not Null

5. password: VARCHAR(100), Not Null

user_id	name	email	phone	password
1	Mohd Siraj	siraj@police.gov	9871110001	pass
2	Rakesh Sharma	rakesh@police.gov	9871110002	pass
3	Anita Deshmukh	anita@police.gov	9871110003	pass
4	Karan Patel	karan@police.gov	9871110004	pass
5	Vikram Singh	vikram@police.gov	9871110005	pass
6	Admin User	admin@system.com	9871110006	adminpass

Table 2.2.2: Admin

• Purpose: Stores administrative details of system administrators.
• Primary Key: admin_id (INT)
• Data Types and Constraints:

1. admin_id: INT, Primary Key, Not Null
2. user_id: INT, Foreign Key, References Users(user_id)
3. office: VARCHAR(100), Not Null

admin_id	user_id	office	contact
1	6	Head Office	9871110006

4. contact: VARCHAR(15), Not Null

Table 2.2.3: Police_Station

• Purpose: Stores information about police stations.
• Primary Key: station_id (INT)
• Data Types and Constraints:

1. station_id: INT, Primary Key, Not Null
2. name: VARCHAR(100), Not Null
3. location: VARCHAR(100), Not Null

station_id	name	location	contact_no
1	Central Police Station	Mumbai	9000011111
2	Andheri Police Station	Mumbai	9000011112
3	Dadar Police Station	Mumbai	9000011113

4. contact_no: VARCHAR(15), Not Null

Table 2.2.4: Police_Officer

• Purpose: Stores details of police officers.
• Primary Key: officer_id (INT)
• Data Types and Constraints:

1. officer_id: INT, Primary Key, Not Null
2. user_id: INT, Foreign Key, References Users(user_id)
3. badge_no: VARCHAR(50), Not Null
4. rank: VARCHAR(50), Not Null

officer_id	user_id	badge_no	rank_1	station_id
101	1	DSP101	DSP	1
102	2	INS102	Inspector	1
103	3	INS103	Inspector	2
104	4	SI104	Sub Inspector	2
105	5	SI105	Sub Inspector	3

5. station_id: INT, Foreign Key, References Police_Station(station_id)

Table 2.2.5: FIR

• Purpose: Stores crime report (FIR) details.
• Primary Key: fir_id (INT)
• Data Types and Constraints:

1. fir_id: INT, Primary Key, Not Null
2. date: DATE, Not Null
3. time: TIME, Not Null

4. description: TEXT, Not Null
5. officer_id: INT, Foreign Key
6. station_id: INT, Foreign Key

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Bank fraud complaint	102	1
2	2025-03-02	12:10:00	Robbery in market	103	2
3	2025-03-03	14:45:00	Cyber fraud case	101	1
4	2025-03-05	09:20:00	Drug trafficking report	105	3
5	2025-03-06	16:30:00	Street assault	104	2

Table 2.2.6: Criminal

• Purpose: Stores details of criminals.
• Primary Key: criminal_id (INT)
• Data Types and Constraints:

1. criminal_id: INT, Primary Key, Not Null
2. name: VARCHAR(100), Not Null
3. gender: VARCHAR(10), Not Null
4. address: VARCHAR(200), Not Null

criminal_id	name	gender	address	remarks
1	Prithviraj Hiray	Male	Pune	Fraud suspect
2	Aditya Gholasgaon	Male	Aurangabad	Robbery suspect
3	Rahul Patil	Male	Nashik	Theft
4	Aman Khan	Male	Mumbai	Cyber crime
5	Rohan Gupta	Male	Delhi	Fraud

5. remarks: TEXT, Null

Table 2.2.7: FIR_Criminal

• Purpose: Resolves many-to-many relationship between FIR and Criminal.
• Primary Key: Composite Key (fir_id, criminal_id)
• Data Types and Constraints:

1. fir_id: INT, Primary Key, Foreign Key

fir_id	criminal_id
1	1
2	2
3	4
1	5
2	7
4	18
3	20
5	22

2. criminal_id: INT, Primary Key, Foreign Key

Table 2.2.8: Criminal_Record

• Purpose: Stores past records of criminals.
• Primary Key: record_id (INT)
• Data Types and Constraints:

1. record_id: INT, Primary Key, Not Null
2. criminal_id: INT, Foreign Key
3. previous_offence: TEXT, Not Null
4. remarks: TEXT, Null

record_id	criminal_id	previous_offence	remarks
1	1	Fraud case 2021	Under investigation
2	2	Robbery case 2022	Repeat offender
3	3	Bike theft	First offence
4	4	Online scam	Under investigation
5	5	Bank fraud	Repeat offender

Table 2.2.9: Court

• Purpose: Stores court details.
• Primary Key: court_id (INT)
• Data Types and Constraints:

1. court_id: INT, Primary Key, Not Null

2. court_name: VARCHAR(100), Not Null
3. judge_name: VARCHAR(100), Not Null

court_id	court_name	judge_name	location
1	Mumbai District Court	Judge R Mehta	Mumbai
2	Sessions Court	Judge S Kulkarni	Mumbai
3	High Court Bench	Judge A Desai	Mumbai

4. location: VARCHAR(100), Not Null

Table 2.2.10: Case_File

• Purpose: Stores case details derived from FIR.
• Primary Key: case_id (INT)
• Data Types and Constraints:

1. case_id: INT, Primary Key, Not Null
2. fir_id: INT, Foreign Key
3. court_id: INT, Foreign Key
4. case_type: VARCHAR(50), Not Null
5. start_date: DATE, Not Null
6. end_date: DATE, Null

case_id	fir_id	court_id	case_type	start_date	end_date	status
1	1	1	Fraud	2025-03-05	NULL	Pending
2	2	2	Robbery	2025-03-06	NULL	Pending
3	3	1	Cyber Crime	2025-03-07	NULL	Investigation
4	4	3	Drug Case	2025-03-08	NULL	Pending

7. status: VARCHAR(50), Not Null

Table 2.2.11: Evidence

• Purpose: Stores evidence related to cases.
• Primary Key: evidence_id (INT)
• Data Types and Constraints:

1. evidence_id: INT, Primary Key, Not Null
2. case_id: INT, Foreign Key
3. description: TEXT, Not Null
4. type: VARCHAR(50), Not Null

evidence_id	case_id	description	type	date_collect...
1	1	Bank transaction records	Document	2025-03-05
2	2	CCTV footage from market	Video	2025-03-06
3	3	Laptop used for fraud	Digital	2025-03-07
4	4	Seized narcotics	Physical	2025-03-08
5	5	Medical report	Document	2025-03-09

5. date_collected: DATE, Not Null

2.3 Creation of Database and Tables – DDL Commands

DDL Commands to create all the tables

```
CREATE DATABASE Criminal_Record_System;
USE Criminal_Record_System;

CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) NOT NULL,
    password VARCHAR(100) NOT NULL
);

CREATE TABLE Police_Station (
    station_id INT PRIMARY KEY,
    name VARCHAR(100),
    location VARCHAR(100),
    contact_no VARCHAR(15)
);

CREATE TABLE Police_Officer (
    officer_id INT PRIMARY KEY,
    user_id INT,
```

```
    badge_no VARCHAR(50),
    rank VARCHAR(50),
    station_id INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (station_id) REFERENCES
Police_Station(station_id)
);
```

2.4 Insertion of Tuples into Tables – DML Commands

```
INSERT INTO Users VALUES
(1,'Mohd Siraj','siraj@police.gov','9871110001','pass'),
(2,'Rakesh Sharma','rakesh@police.gov','9871110002','pass'),
(3,'Anita Deshmukh','anita@police.gov','9871110003','pass'),
(4,'Karan Patel','karan@police.gov','9871110004','pass'),
(5,'Vikram Singh','vikram@police.gov','9871110005','pass'),
(6,'Admin User','admin@system.com','9871110006','adminpass');
```

Example Table Output

User Table

user_id	name	Email	phone	password
1	Mohd Siraj	siraj@police.gov	9871110001	pass
2	Rakesh Sharma	rakesh@police.gov	9871110002	pass
3	Anita Deshmukh	anita@police.gov	9871110003	pass
4	Karan Patel	karan@police.gov	9871110004	pass
5	Vikram Singh	vikram@police.gov	9871110005	pass
6	Admin User	admin@system.com	9871110006	Adminpass

CHAPTER 3

Complex queries based on the concepts of constraints, sets, joins, views, triggers and cursors

3.1 Adding Constraints and Queries Based on Constraints

Question:
Add a constraint to ensure case status can only be ‘Pending’, ‘Solved’, or ‘Investigation’.

SQL Statement:

```
ALTER TABLE Case_File
ADD CONSTRAINT chk_status
CHECK (status IN ('Pending','Solved','Investigation'));
```

Output:
Constraint added successfully. Invalid values will be rejected.

3.2 Queries Based on Aggregate Functions

Question:
Find total number of FIRs registered in each police station.

SQL Statement:

```
SELECT station_id, COUNT(*) AS total_FIRs
FROM FIR
GROUP BY station_id;
```

Output:

station_id	total_FIRs
1	2
2	2
3	1

3.3 Complex Queries Based on Sets

Question:
Find criminals who are involved in FIR cases (using INTERSECT concept).

SQL Statement:

```
SELECT criminal_id FROM Criminal
INTERSECT
SELECT criminal_id FROM FIR_Criminal;
```

Output:

criminal_id
1
2
4

3.4 Complex Queries Based on Subqueries

Question:
Find names of criminals involved in FIR with ID = 1.

SQL Statement:

```
SELECT name
FROM Criminal
WHERE criminal_id IN
(
SELECT criminal_id
FROM FIR_Criminal
WHERE fir_id = 1
);
```

Output:

name
Prithviraj Hiray
Rohan Gupta

3.5 Complex Queries Based on Joins

Question:
Display crime description along with officer name and station name.

SQL Statement:

```
SELECT f.description, u.name AS officer_name, p.name AS
station_name
FROM FIR f
JOIN Police_Officer po ON f.officer_id = po.officer_id
JOIN Users u ON po.user_id = u.user_id
JOIN Police_Station p ON f.station_id = p.station_id;
```

Output:

description	officer_name	station_name
Bank fraud complaint	Mohd Siraj	Central Station

3.6 Complex Queries Based on Views

Question:
Create a view to display all pending cases.

SQL Statement:

```
CREATE VIEW Pending_Cases AS
SELECT case_id, case_type, status
FROM Case_File
WHERE status = 'Pending';
SELECT * FROM Pending_Cases;
```

Output:

case_id	case_type	status
1	Fraud	Pending
2	Robbery	Pending

3.7 Complex Queries Based on Triggers

Question:
Automatically set default status as ‘Pending’ when a new case is inserted.

SQL Statement:

```
CREATE TRIGGER set_default_status
BEFORE INSERT ON Case_File
FOR EACH ROW
SET NEW.status = 'Pending';
```

Output:
When inserting a new case without status, it will automatically be set to ‘Pending’.

3.8 Complex Queries Based on Cursors

Question:
Display all criminal names using a cursor.

SQL Statement:

```
DECLARE done INT DEFAULT FALSE;
DECLARE cname VARCHAR(100);

DECLARE cur CURSOR FOR SELECT name FROM Criminal;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN cur;

read_loop: LOOP
    FETCH cur INTO cname;
    IF done THEN
        LEAVE read_loop;
    END IF;
    SELECT cname;
END LOOP;

CLOSE cur;
```

Output:
List of all criminal names displayed one by one.

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

4.1 Analyse the Pitfalls in Relations

Before applying normalization, the database structure contained a single large relation storing FIR, criminal, police, and case details together. This resulted in redundancy and anomalies.

Table 4.1: Crime Report UNF (Before Normalization)

fir_id	criminal_name	officer_name	station	case_type	evidence
1	A, B	Officer1	Station1	Theft	Doc1
2	C	Officer2	Station2	Fraud	Doc2

Pitfalls Identified

When all crime-related data such as FIR, criminal, officer, and case details are stored in a single unnormalized table, several issues arise:

1. Repeating Groups (Violation of Atomicity)
Certain attributes contain multiple values in a single column instead of atomic values.

Example:
A single FIR may involve multiple criminals stored as:

criminal_name = 'A, B, C'

This makes querying, updating, and searching difficult since relational databases require atomic values.

2. Data Redundancy
The same information is stored repeatedly across multiple rows, increasing storage usage and causing inconsistency.

Example:
If multiple FIRs are handled by the same officer:

officer_name = 'DSP Mohd Siraj’

is stored repeatedly for every FIR entry. If there are 100 FIRs, the same name is stored 100 times.

3. Update Anomaly
Updating a value requires changes in multiple rows, which may lead to inconsistencies if some rows are not updated.

Example:
If the station name changes from:

'Central Station' → 'Chennai Central Station'

it must be updated in all rows. Missing even one row leads to inconsistent data.

4. Insertion Anomaly
It becomes difficult to insert data without the presence of other related data.

Example:
A criminal cannot be added to the system unless an FIR is already registered.
This means independent data (criminal details) cannot be stored separately.

5. Deletion Anomaly
Deleting one piece of data may unintentionally remove other important data.

Example:
If an FIR record is deleted, the associated criminal details may also be lost, even though the criminal might be involved in other cases.

These issues highlight the need for normalization to ensure data integrity, reduce redundancy, and improve database efficiency.

4.2 First Normal Form (1NF)

4.2.1 Identify Dependency

Functional Dependencies identified:

fir_id → officer_name, station, case_type

The attribute *criminal_name* contains multi-valued data.

4.2.2 Apply Normalization to 1NF

The repeating groups are removed.

Table 4.2: FIR (After 1NF)

fir_id	officer_id	station_id	case_type
1	101	1	Theft
2	102	2	Fraud

Table 4.3: FIR_Criminal

fir_id	**criminal_id**
1	**A**
1	**B**
2	**C**

After applying 1NF, all attributes contain atomic values.

4.3 Second Normal Form (2NF)

4.3.1 Identify Dependency

Partial dependency identified:

criminal_id → criminal_name, address

4.3.2 Apply Normalization to 2NF

Separate criminal details into a new table.

Table 4.4: Criminal

criminal_id	name	address
A	Rahul	Chennai
B	Arjun	Mumbai
C	Karan	Pune

Table 4.5: FIR_Criminal (Updated)

fir_id	criminal_id
1	A
1	B
2	C

After applying 2NF, partial dependency is removed.

4.4 Third Normal Form (3NF)

4.4.1 Identify Dependency

Transitive dependency identified:

officer_id → station_id

station_id → station_name

Thus:

officer_id → station_name

4.4.2 Apply Normalization to 3NF

Separate station details.

Table 4.6: Police_Station

station_id	station_name	location
1	Central	Chennai
2	North	Mumbai

Table 4.7: Police_Officer

officer_id	name	station_id

101	DSP Mohd Siraj	1
102	Officer2	2

Updated FIR Table

fir_id	officer_id	station_id	case_type
1	101	1	Theft

After applying 3NF, transitive dependency is removed.

4.5 Boyce-Codd Normal Form (BCNF)

4.5.1 Identify Dependency

Dependency:

officer_id → station_id

Officer determines station but must be handled properly as a key.

4.5.2 Apply Normalization to BCNF

Ensure every determinant is a candidate key.

Table 4.8: FIR (BCNF)

fir_id	officer_id	case_type

After BCNF, every determinant becomes a candidate key.

4.6 Fourth Normal Form (4NF)

4.6.1 Identify Dependency

Multivalued dependency:

criminal_id →→ previous_offence

criminal_id →→ remarks

4.6.2 Apply Normalization to 4NF

Split into separate tables.

Table 4.9: Criminal_Record

record_id	criminal_id	previous_offence	remarks

After applying 4NF, multivalued dependency is removed.

4.7 Fifth Normal Form (5NF)

4.7.1 Identify Dependency

Join dependency:

FIR ↔ Criminal ↔ Case

4.7.2 Apply Normalization to 5NF

Split into smaller relations.

Table 4.10: FIR_Criminal

fir_id	criminal_id

Table 4.11: Case_File

case_id	fir_id	case_type	status

After applying 5NF, join dependency is removed and database achieves maximum normalization.

Conclusion

After applying normalization from 1NF to 5NF, redundancy is eliminated, anomalies are removed, and data integrity is ensured. The final database design is efficient, scalable, and suitable for real-world crime management systems.

CHAPTER 5

IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

A transaction is a sequence of one or more SQL operations executed as a single logical unit of work. In the Crime Report Management System, transactions ensure that operations such as inserting FIR records, updating case status, managing criminal data, and storing evidence are executed reliably and consistently.

Transactions help maintain database integrity even when multiple users access the system simultaneously or when failures occur.

5.1.1 Properties

Transactions follow ACID properties to ensure reliability:

1. Atomicity
All operations in a transaction are completed or none are executed.

SQL Statement:

START TRANSACTION;

INSERT INTO FIR

VALUES (10,'2025-04-01','10:30:00','Robbery Case',101,1);

ROLLBACK;

Output:

fir_id	date	time	description	officer_id	station_id
NULL	NULL	NULL	NULL	NULL	NULL

2. Consistency
Ensures database remains valid.

Example:

ALTER TABLE Case_File

ADD CONSTRAINT chk_status

CHECK (status IN ('Pending','Solved','Investigation'));

START TRANSACTION; INSERT INTO Case_File VALUES (10,1,1,'Fraud','2025-04-01',NULL,'InvalidStatus'); ROLLBACK;



3. Isolation
Transactions run independently without interference.
Example: Multiple users updating the same FIR will not conflict due to locking mechanisms.

SQL Statement (Transaction 1):

START TRANSACTION;

SELECT * FROM FIR

WHERE fir_id = 1

FOR UPDATE;

SQL Statement (Transaction 2):

START TRANSACTION;

UPDATE FIR

SET description = 'Updated by T2'

WHERE fir_id = 1;

Output:

Transaction 1

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Bank fraud complaint	102	1
NULL	NULL	NULL	NULL	NULL	NULL

Transaction 2

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Updated by T2	102	1
NULL	NULL	NULL	NULL	NULL	NULL

4. Durability
Committed data is permanently saved.

SQL Statement:

START TRANSACTION;

UPDATE Case_File SET status = 'Solved' WHERE case_id = 1;

COMMIT;

SELECT case_id, status FROM Case_File WHERE case_id = 1;

Output:

case_id	status
1	Solved
NULL	NULL

5.1.2 Transaction States

A transaction goes through:
• Active → executing
• Partially Committed → last statement executed

• Committed → saved permanently
• Failed → error occurred
• Aborted → rolled back

5.2 Transaction Control Language (TCL)

TCL commands manage transactions.

5.2.1 Savepoint

Creates a checkpoint.

SQL Statement:

START TRANSACTION;

UPDATE Criminal

SET address = 'Updated Address'

WHERE criminal_id = 1;

SAVEPOINT sp1;

5.2.2 Commit

Saves changes permanently.

SQL Statement:

COMMIT;

5.2.3 Rollback

Undo changes.

SQL Statement:

ROLLBACK;

5.3 Transactions Implementation

Transaction 1 — Insert FIR

START TRANSACTION;

INSERT INTO FIR VALUES (11,'2025-04-10','11:00:00','Cyber Crime',102,2);

SAVEPOINT sp1;

INSERT INTO FIR_Criminal VALUES (11,3);

ROLLBACK TO sp1;

COMMIT;

SELECT * FROM FIR;

Output:

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Updated by T2	102	1
2	2025-03-02	12:10:00	Robbery in market	103	2
3	2025-03-03	14:45:00	Cyber fraud case	101	1
4	2025-03-05	09:20:00	Drug trafficking report	105	3
5	2025-03-06	16:30:00	Street assault	104	2
11	2025-04-10	11:00:00	Cyber Crime	102	2
NULL	NULL	NULL	NULL	NULL	NULL

Transaction 2 — Insert Case

START TRANSACTION;

INSERT INTO Case_File VALUES (11,11,1,'Cyber','2025-04-10',NULL,'Pending');

SAVEPOINT sp2;

UPDATE Case_File

SET status = 'Investigation'

WHERE case_id = 11;

COMMIT;

SELECT * FROM Case_File;

Output:

case_id	fir_id	court_id	case_type	start_date	end_date	status
1	1	1	Fraud	2025-03-05	NULL	Solved
2	2	2	Robbery	2025-03-06	NULL	Pending
3	3	1	Cyber Crime	2025-03-07	NULL	Investigation
4	4	3	Drug Case	2025-03-08	NULL	Pending
5	5	2	Assault	2025-03-09	NULL	Pending
11	11	1	Cyber	2025-04-10	NULL	Investigation
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Transaction 3 — Update Case Status

START TRANSACTION;

UPDATE Case_File

SET status = 'Solved'

WHERE case_id = 2;

SAVEPOINT sp3;

COMMIT;

SELECT * FROM Case_File;

Output:

case_id	fir_id	court_id	case_type	start_date	end_date	status
1	1	1	Fraud	2025-03-05	NULL	Solved
2	2	2	Robbery	2025-03-06	NULL	Solved
3	3	1	Cyber Crime	2025-03-07	NULL	Investigation
4	4	3	Drug Case	2025-03-08	NULL	Pending
5	5	2	Assault	2025-03-09	NULL	Pending
11	11	1	Cyber	2025-04-10	NULL	Investigation
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Transaction 4 — Insert Evidence

START TRANSACTION;

INSERT INTO Evidence VALUES (11,11,'Laptop Logs','Digital','2025-04-10');

SAVEPOINT sp4;

ROLLBACK TO sp4;

COMMIT;

SELECT * FROM Evidence;

Output:

evidence_id	case_id	description	type	date_collect...
1	1	Bank transaction records	Document	2025-03-05
2	2	CCTV footage from market	Video	2025-03-06
3	3	Laptop used for fraud	Digital	2025-03-07
4	4	Seized narcotics	Physical	2025-03-08
5	5	Medical report	Document	2025-03-09
11	11	Laptop Logs	Digital	2025-04-10
NULL	NULL	NULL	NULL	NULL

Transaction 5 — Delete Criminal Record

START TRANSACTION;

DELETE FROM Criminal_Record

```
WHERE record_id = 3;

SAVEPOINT sp5;

ROLLBACK TO sp5;

COMMIT;

SELECT * FROM Criminal_Record;
```

Output:

record_id	criminal_id	previous_offence	remarks
1	1	Fraud case 2021	Under investigation
2	2	Robbery case 2022	Repeat offender
4	4	Online scam	Under investigation
5	5	Bank fraud	Repeat offender
NULL	NULL	NULL	NULL

5.4 Concurrency Control

Concurrency control manages multiple users accessing the database simultaneously while maintaining consistency.

In this project, locking mechanisms are used.

5.4.1 Concurrency Control Algorithms

Locking-Based Concurrency Control ensures only one transaction modifies data at a time.

a. Row-Level Locking

```
START TRANSACTION;

SELECT *

FROM FIR
```

WHERE fir_id = 1

FOR UPDATE;

UPDATE FIR

SET description = 'Updated Case'

WHERE fir_id = 1;

COMMIT;

Output:

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Updated by T2	102	1
NULL	NULL	NULL	NULL	NULL	NULL

b. Table-Level Locking

LOCK TABLE FIR WRITE;

UPDATE FIR

SET description = 'Table Lock Update'

WHERE fir_id = 2;

UNLOCK TABLES;

Lock Modes

Lock Mode	Description
ROW SHARE	Allows multiple transactions to read rows simultaneously without blocking each other
ROW EXCLUSIVE	Allows a transaction to update rows but restricts other conflicting updates
SHARE	Allows read-only access; other transactions can also read but cannot modify
SHARE ROW EXCLUSIVE	Allows read operations but restricts both read and write conflicts partially
EXCLUSIVE	Full table lock; no other transaction can read or write the table
INTENTION SHARE	Indicates intention to acquire a shared lock on rows (used internally by DBMS)
INTENTION EXCLUSIVE	Indicates intention to acquire an exclusive lock on rows
READ LOCK	Allows only read operations; prevents write operations

WRITE LOCK	Allows both read and write operations for the transaction but blocks others
UPDATE LOCK	Used to prevent deadlocks when updating data; allows reading but restricts other updates

c. COMMIT – Release All Locks

COMMIT is used to permanently save changes such as updating FIR or case data. It releases all locks.

```
COMMIT;
```

d. ROLLBACK – Undo Changes & Release Locks

ROLLBACK is used to undo incorrect operations and restore previous data.

```
ROLLBACK;
```

5.4.2 Concurrency Control Example

Transaction T1:

```
START TRANSACTION;
SELECT * FROM FIR
WHERE fir_id = 3
FOR UPDATE;
UPDATE FIR
```

SET description = 'T1 Update'

WHERE fir_id = 3;

COMMIT;

Output:

fir_id	date	time	description	officer_id	station_id
3	2025-03-03	14:45:00	Cyber fraud case	101	1
NULL	NULL	NULL	NULL	NULL	NULL

Transaction T2:

START TRANSACTION;

SELECT * FROM FIR

WHERE fir_id = 3

FOR UPDATE;

UPDATE FIR

SET description = 'T2 Update'

WHERE fir_id = 3

COMMIT;

Output:

fir_id	date	time	description	officer_id	station_id
3	2025-03-03	14:45:00	T1 Update	101	1
NULL	NULL	NULL	NULL	NULL	NULL

5.5 Recovery Mechanisms

Recovery mechanisms ensure that the database can be restored to a consistent state after failure.

Types of Failures

- Transaction failure
- System crash
- Hardware failure

Recovery Techniques

- Log-based recovery
- Checkpointing
- Backup and restore

Conclusion

The implementation of transactions, concurrency control, and recovery mechanisms ensures that the Crime Report Management System maintains data consistency, reliability, and integrity even in multi-user environments. These mechanisms are essential for handling real-world database operations efficiently and securely.

CHAPTER 6

FRONT-END AND BACK-END CODE OF CRIME REPORT MANAGEMENT SYSTEM

This chapter presents the implementation details of the Criminal Record Management System (CRMS), including the frontend interface, backend logic, and database connectivity. The system follows a full-stack architecture where the frontend interacts with backend APIs, and the backend communicates with a relational database to ensure efficient data storage, retrieval, and consistency.

The application is designed to provide a secure, scalable, and user-friendly environment for managing criminal records, FIRs, case files, evidence, court hearings, and user roles.

Technology Stack

Frontend: React.js (with Vite) – used for building dynamic and responsive user interfaces
Backend: Node.js with Express.js – used to handle API requests and server-side logic
Database: MySQL – used for structured storage and management of data
Styling: CSS / modern UI components – used to design an interactive and clean interface

6.1 Front-End Module Codes

The frontend of the system is built using a component-based architecture in React, ensuring modularity, reusability, and efficient state management. Routing is handled using React Router, and authentication is managed using context-based state handling.

The main entry point of the frontend is App.jsx, which defines all routes and navigation logic.

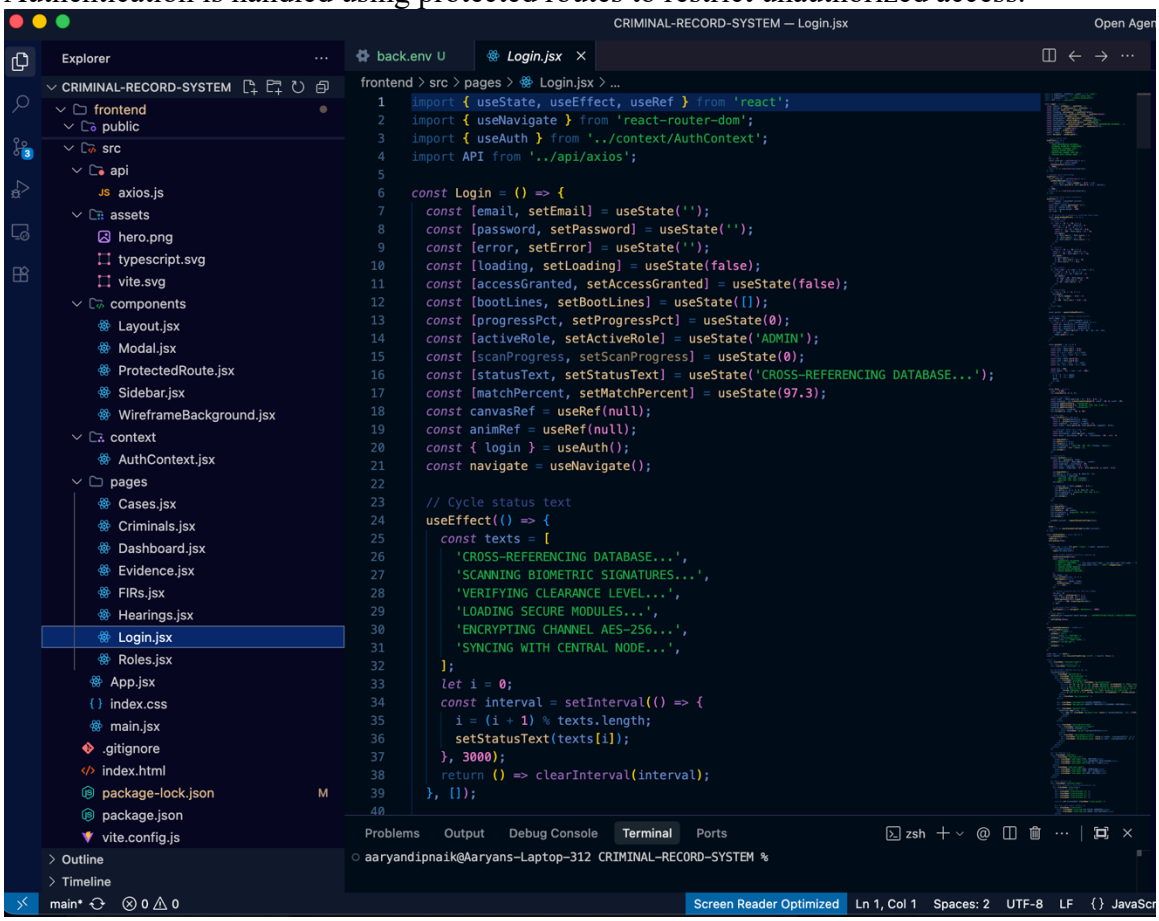
Key features implemented in frontend:

- Role-based login system (Admin, Officer, Clerk)
- Protected routes using authentication context
- Dynamic dashboards with real-time data
- Modular pages for different functionalities

Main Modules:

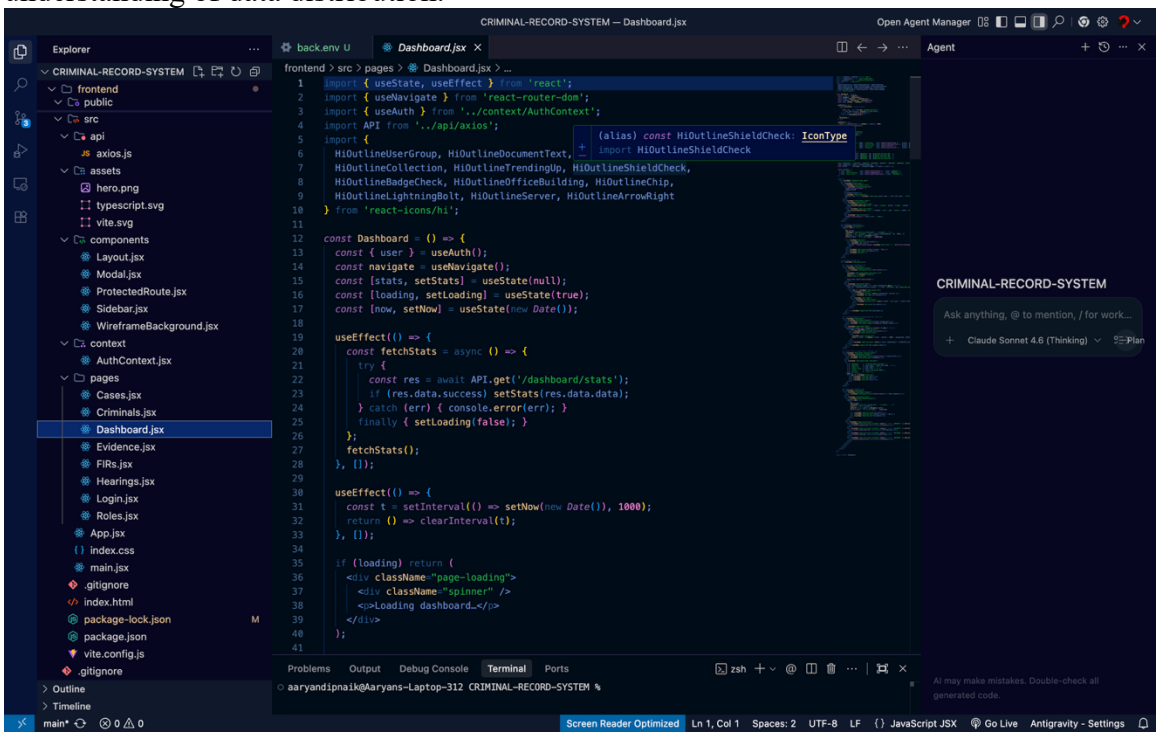
Login Module

The login page allows users to securely access the system based on their role. Authentication is handled using protected routes to restrict unauthorized access.



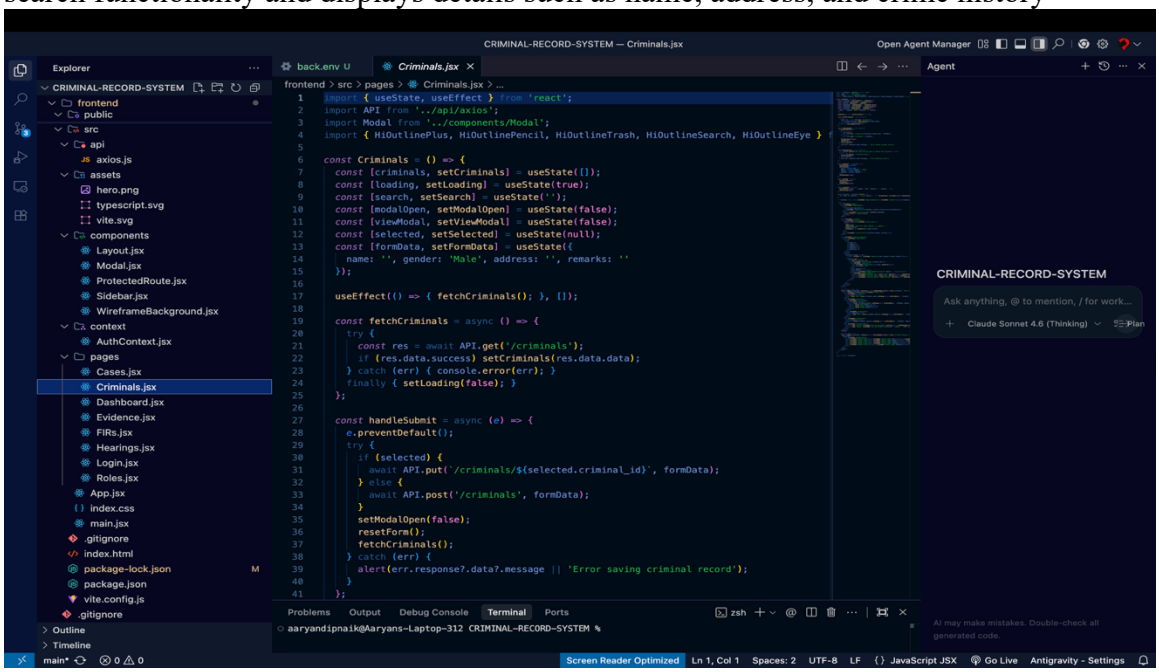
Dashboard Module

The dashboard provides a summary of system data such as number of criminals, FIR records, case files, and evidence. It also includes visual summaries for better understanding of data distribution.



Criminal Records Module

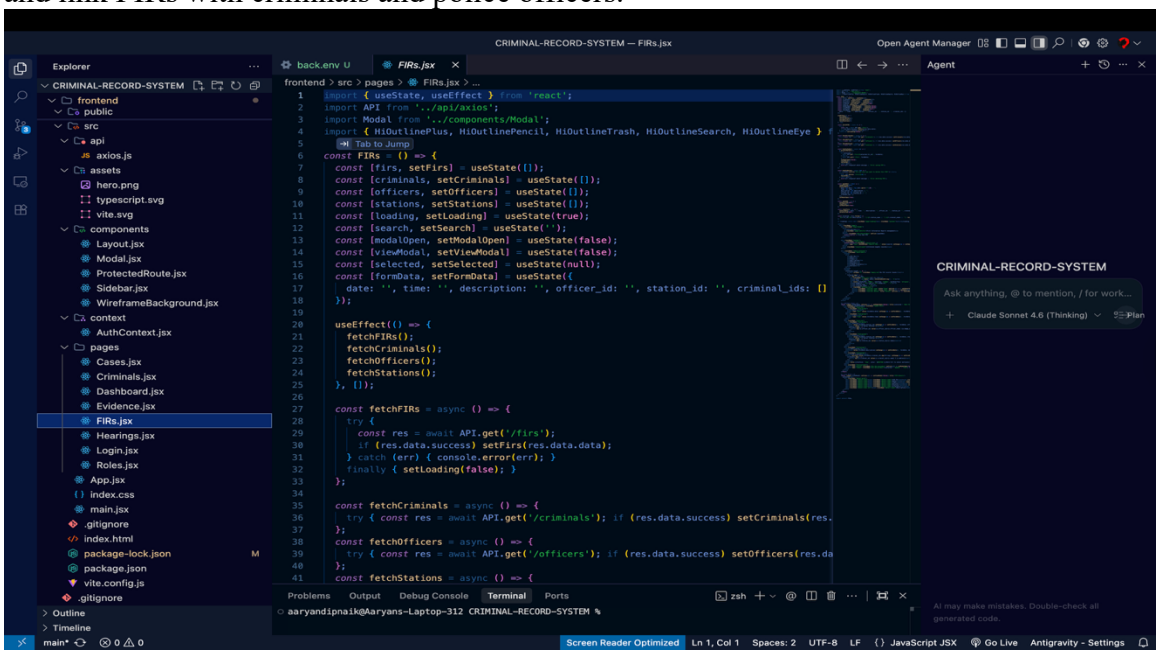
This module allows users to add, view, update, and delete criminal records. It includes search functionality and displays details such as name, address, and crime history



.

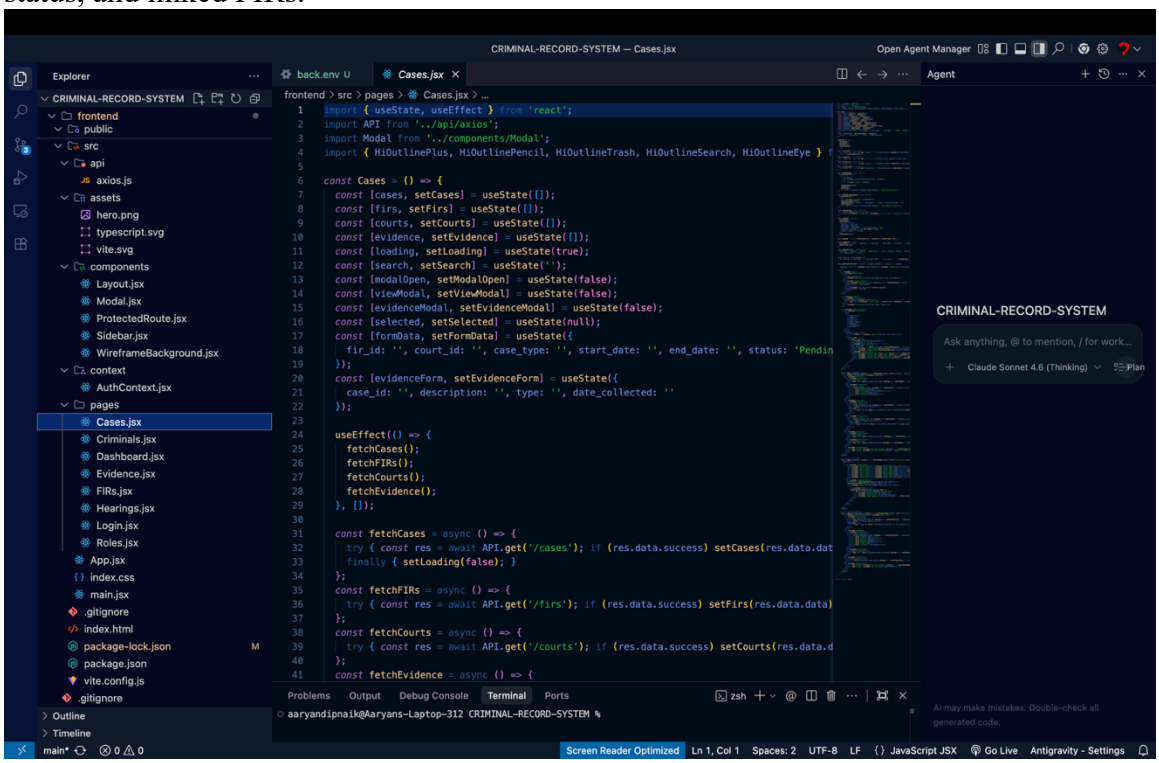
FIR Records Module

This module manages First Information Reports. Users can register FIRs, view records, and link FIRs with criminals and police officers.



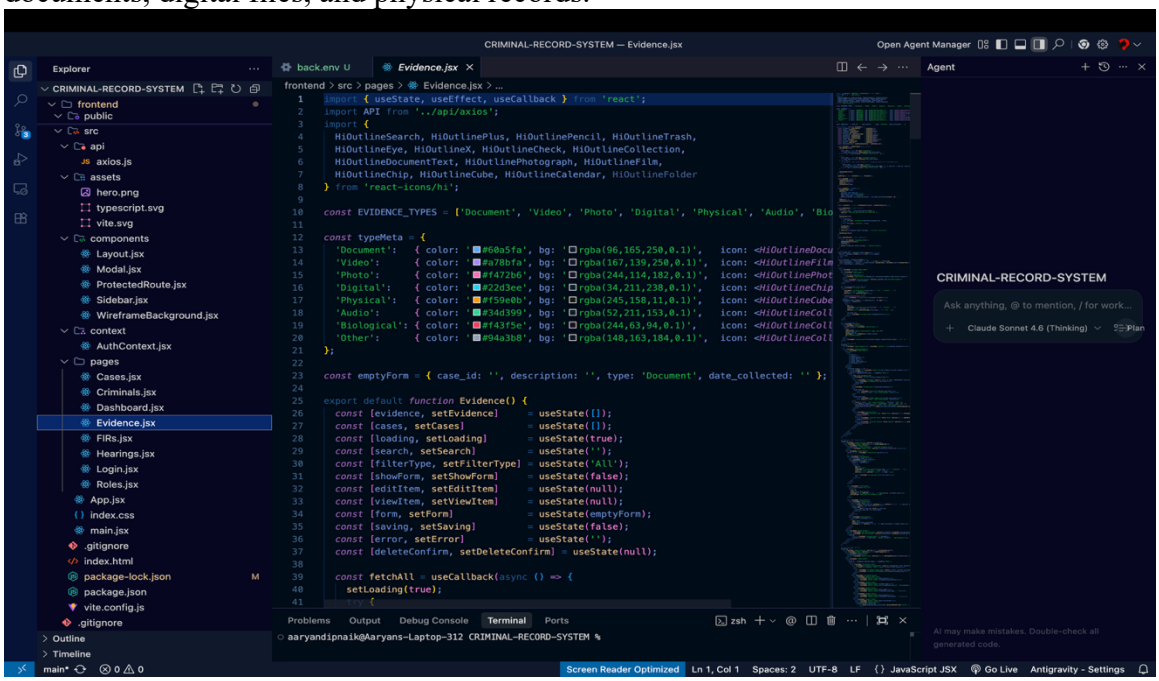
Case Files Module

This module tracks ongoing and completed cases. It includes case ID, court details, status, and linked FIRs.



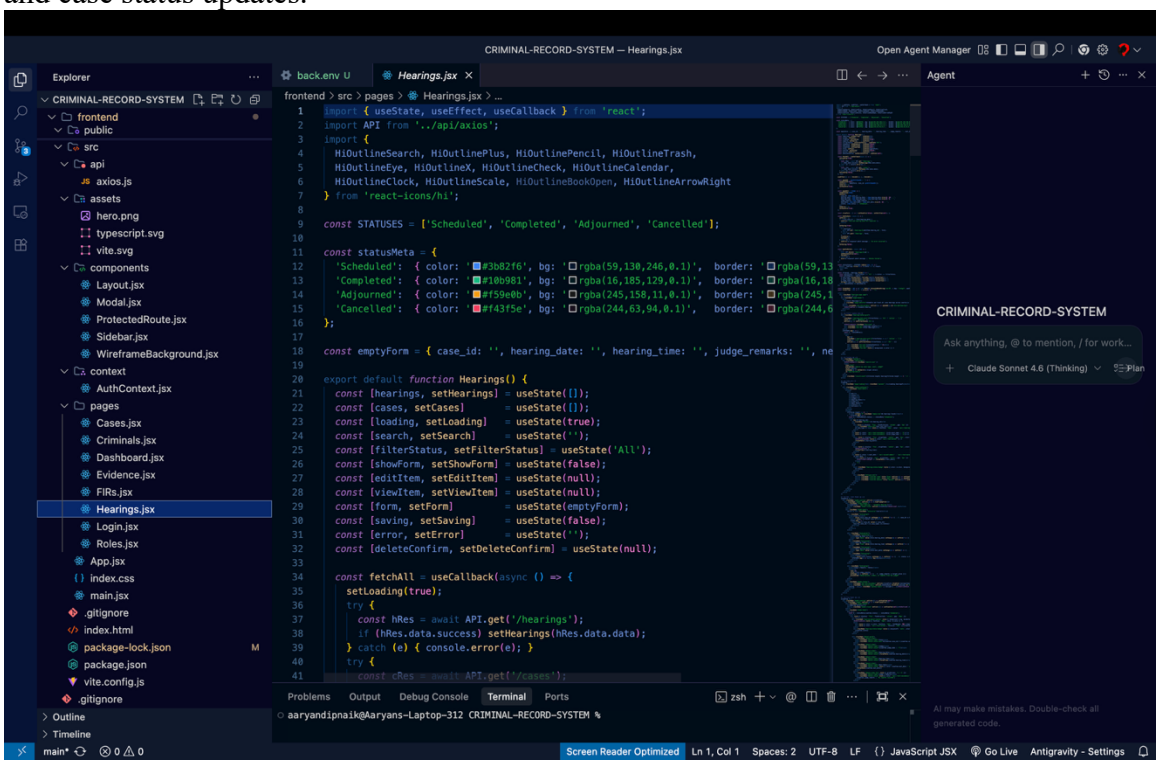
Evidence Module

This module stores and manages evidence related to cases. Evidence types include documents, digital files, and physical records.



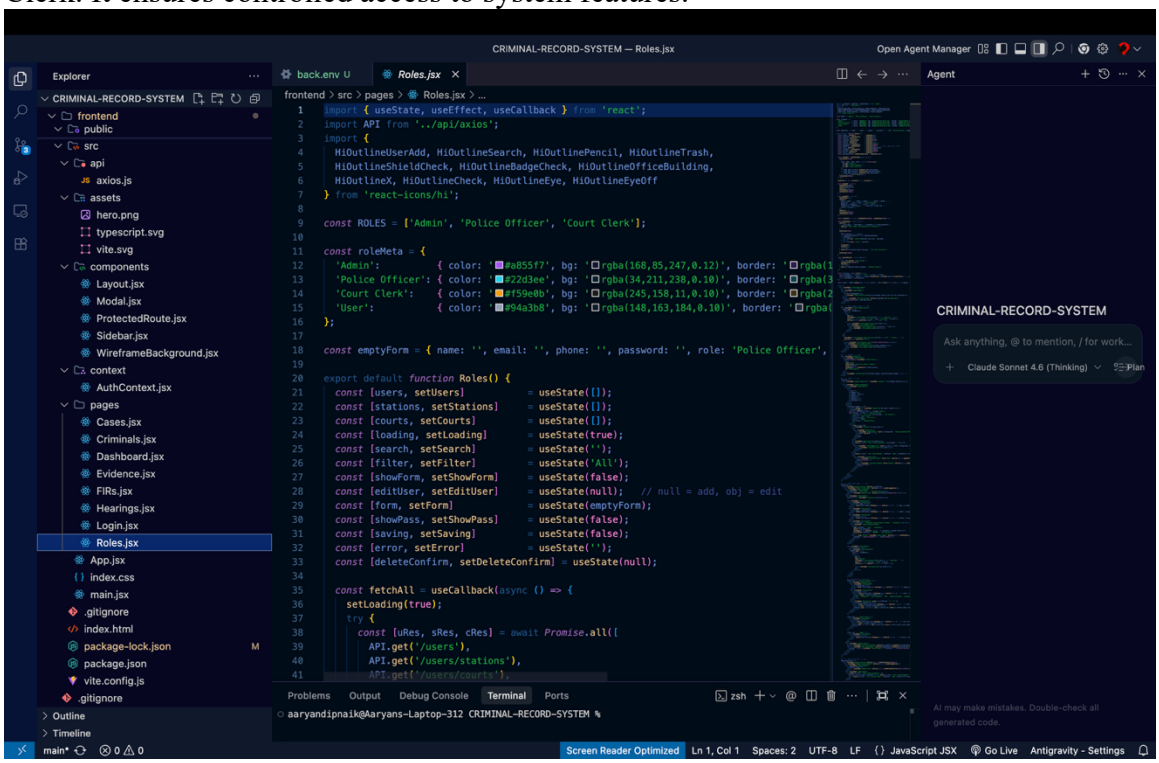
Hearings Module

This module schedules and tracks court hearings. It includes judge details, hearing dates, and case status updates.



Role Management Module

This module manages users and assigns roles such as Admin, Police Officer, and Court Clerk. It ensures controlled access to system features.

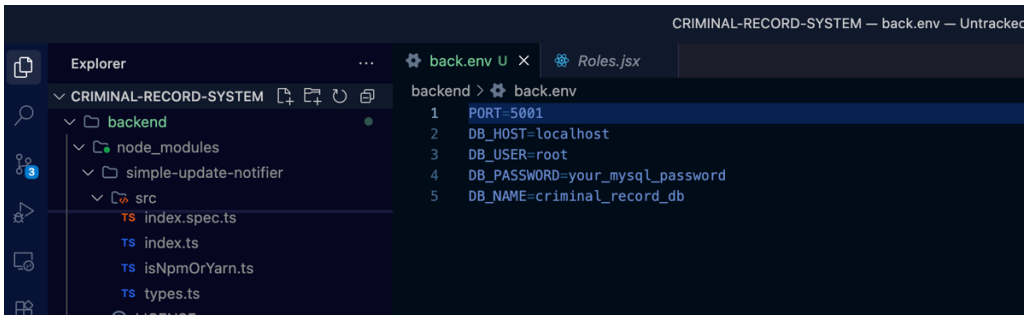


6.2 Database Connectivity

The backend is implemented using Node.js and Express.js, which handle API requests and database interactions. The backend acts as a bridge between the frontend and the database.

Database connectivity is established using MySQL, ensuring structured data storage and efficient query execution.

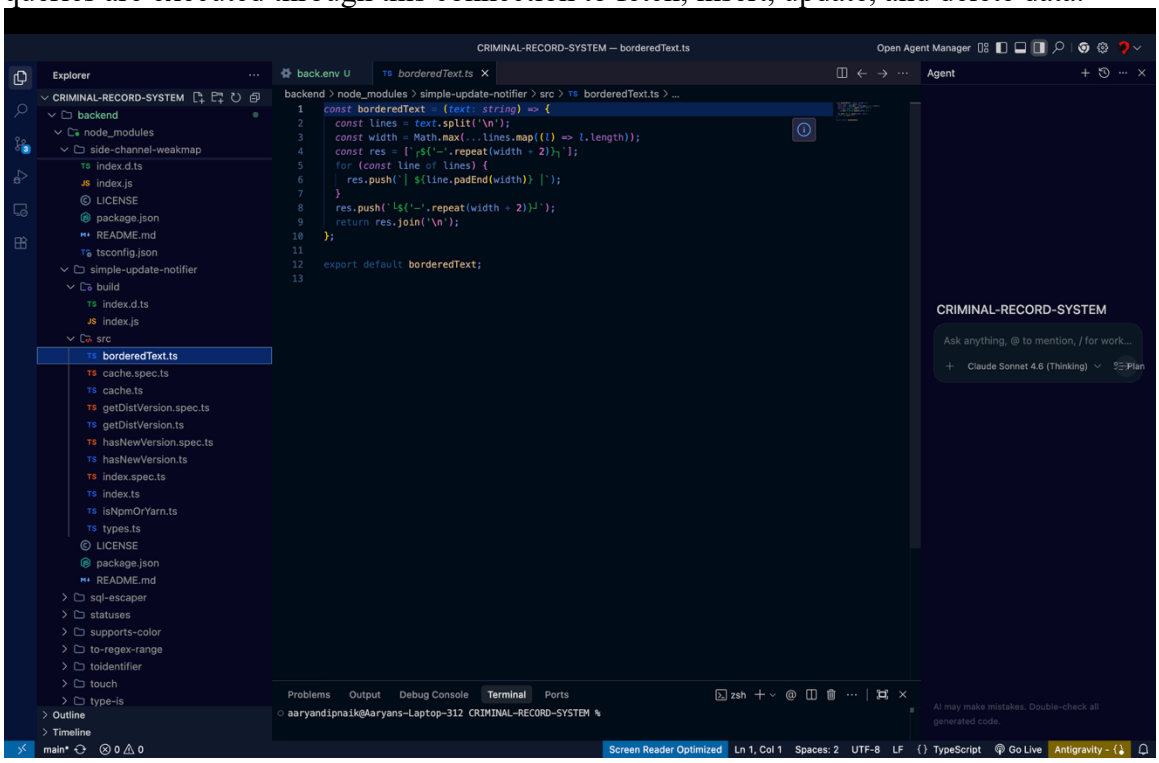
Environment Configuration:



Backend Features:

• REST API architecture for communication between frontend and backend
• CRUD operations for all modules (Criminals, FIRs, Cases, Evidence, Hearings)
• Role-based access control
• Error handling and validation

Database Connection
The backend connects to the MySQL database using a connection configuration file. All queries are executed through this connection to fetch, insert, update, and delete data.



API Handling
All API routes are defined in the backend and handle requests such as:

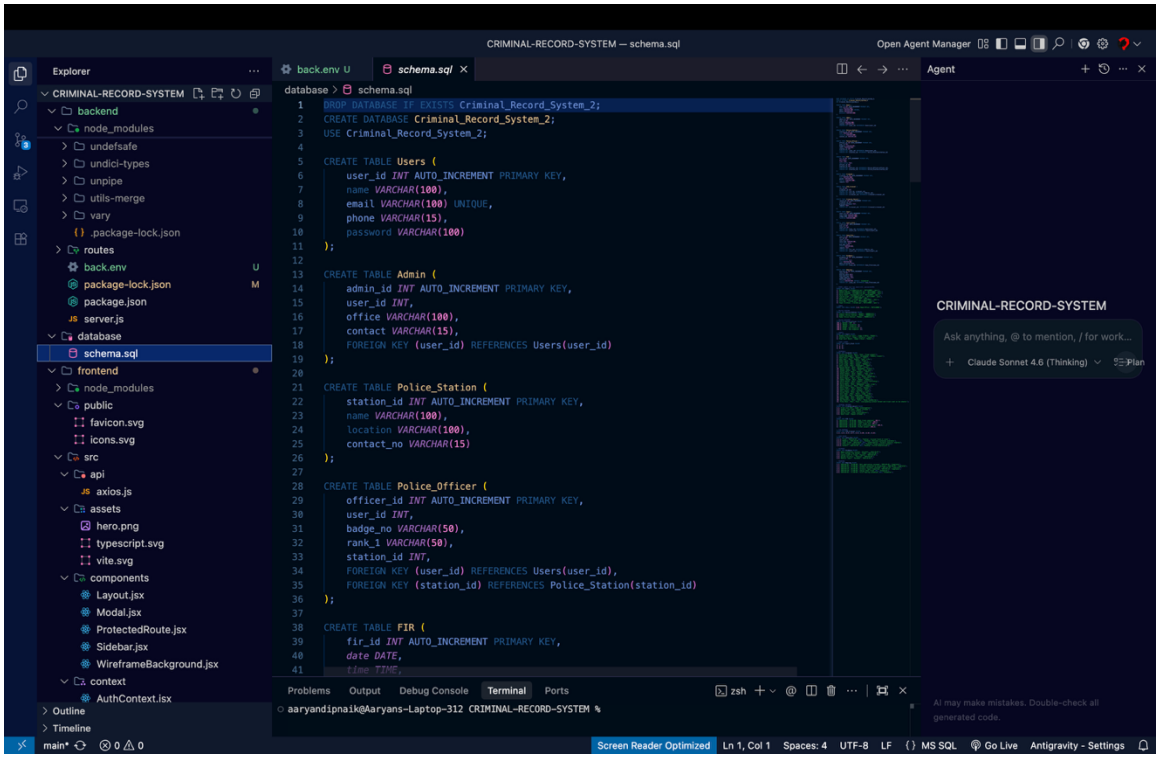
GET – retrieve data from database
POST – insert new records
PUT – update existing records
DELETE – remove records

Example Functionalities:

• Fetch all criminals from database
• Register new FIR records
• Update case status
• Store and retrieve evidence data
• Schedule and manage court hearings

Database Schema
The database consists of multiple related tables such as:

• Criminals
• FIR Records
• Case Files
• Evidence
• Hearings
• Users and Roles



Foreign key relationships are used to maintain referential integrity between tables.

Conclusion

The frontend and backend integration of the Criminal Record Management System ensures efficient handling of law enforcement data through a structured and scalable approach. The use of modern web technologies and relational database design enables secure data management, real-time updates, and improved usability. This implementation demonstrates practical application of full-stack development along with core DBMS concepts such as data integrity, normalization, and relational mapping.

GitHub Repository Link:
https://github.com/Shantanu0913/CRIMINAL-RECORD-SYSTEM

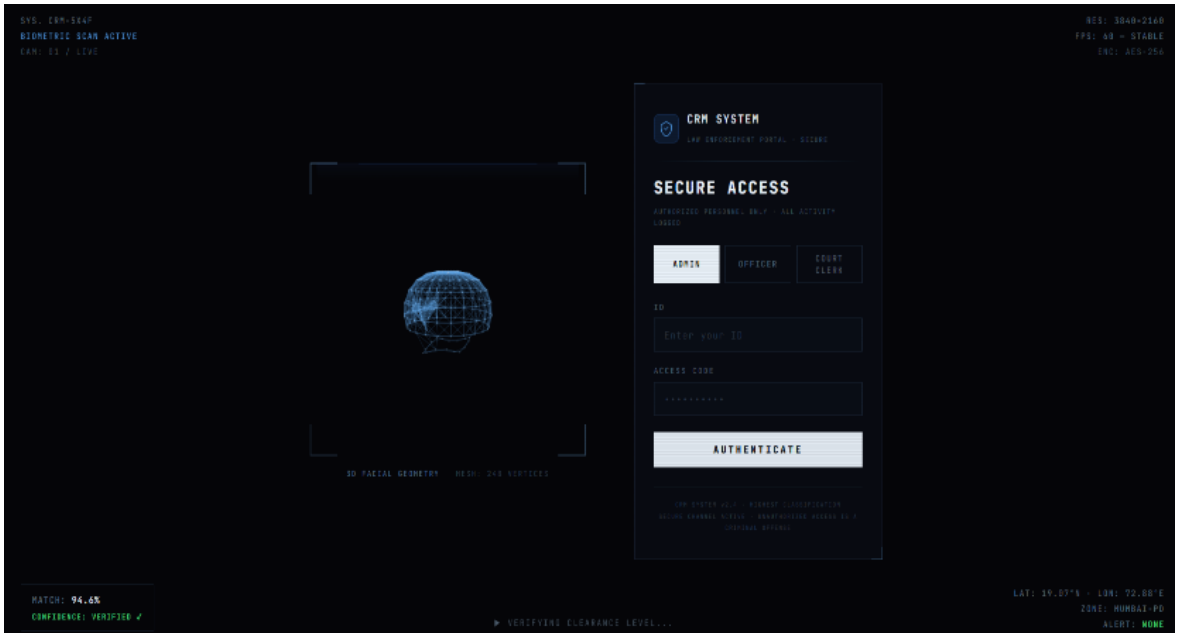
CHAPTER 7

CHAPTER 7 SYSTEM INTERFACE AND OUTPUT SCREENS OF CRIME REPORT MANAGEMENT SYSTEM

7.1 Introduction

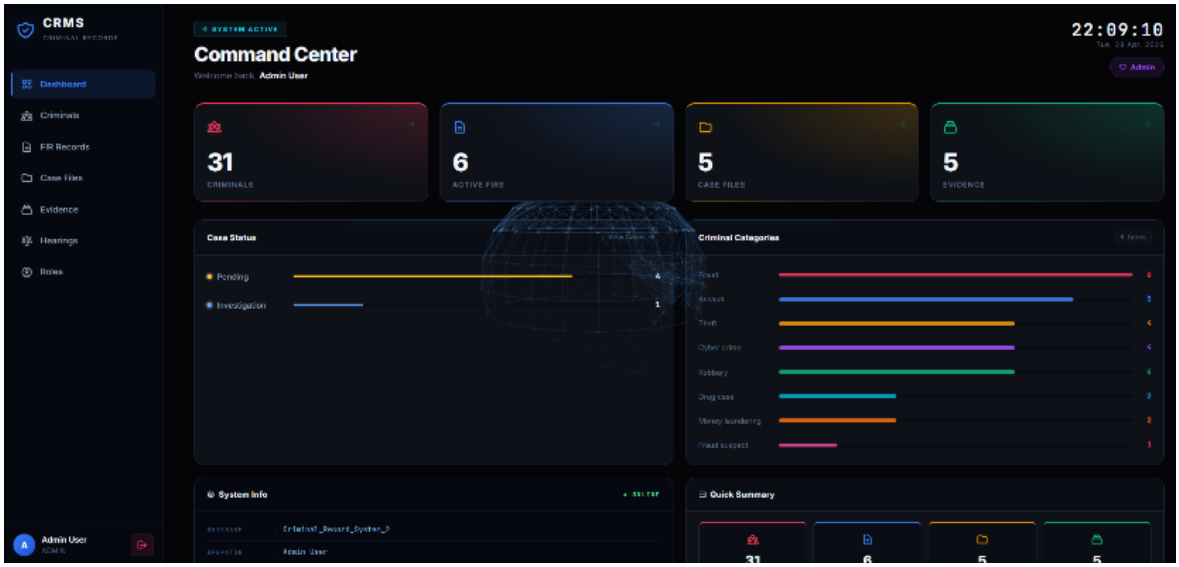
The Crime Report Management System provides a modern and interactive user interface designed for efficient handling of criminal records, FIRs, case files, evidence, court hearings, and user roles. The system interface is developed using React.js and presents data in a structured and visually appealing manner. This chapter showcases the different output screens of the system and explains their functionality.

7.2 Login and Secure Access Module



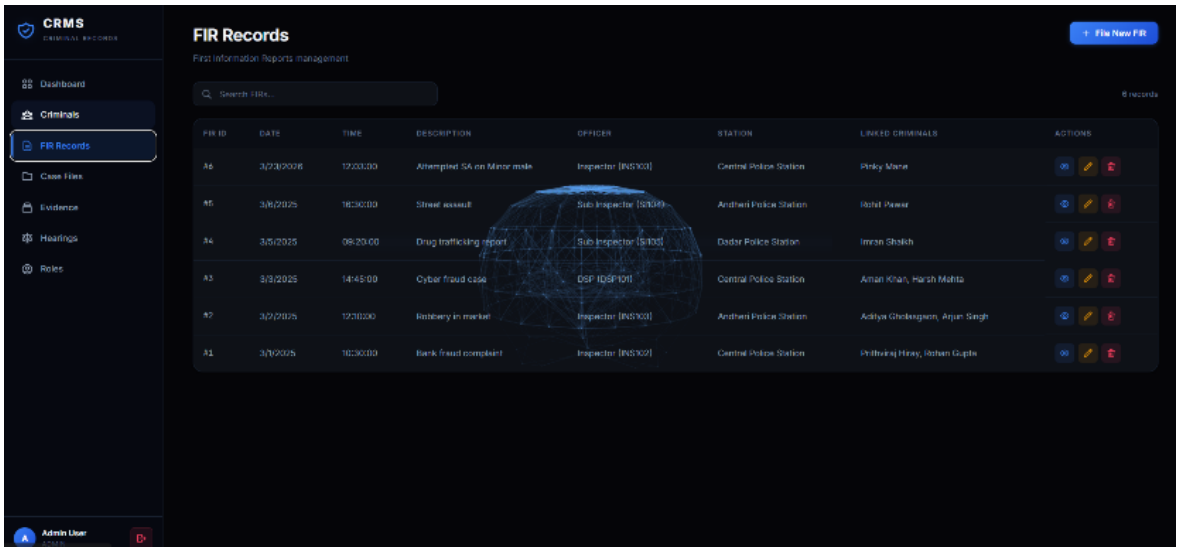
The system begins with a secure login interface where users authenticate themselves based on their role such as Admin, Police Officer, or Court Clerk. The login screen ensures controlled access to the system by validating user credentials. This enhances security and prevents unauthorized usage of sensitive criminal data.

7.3 Dashboard (Command Center)



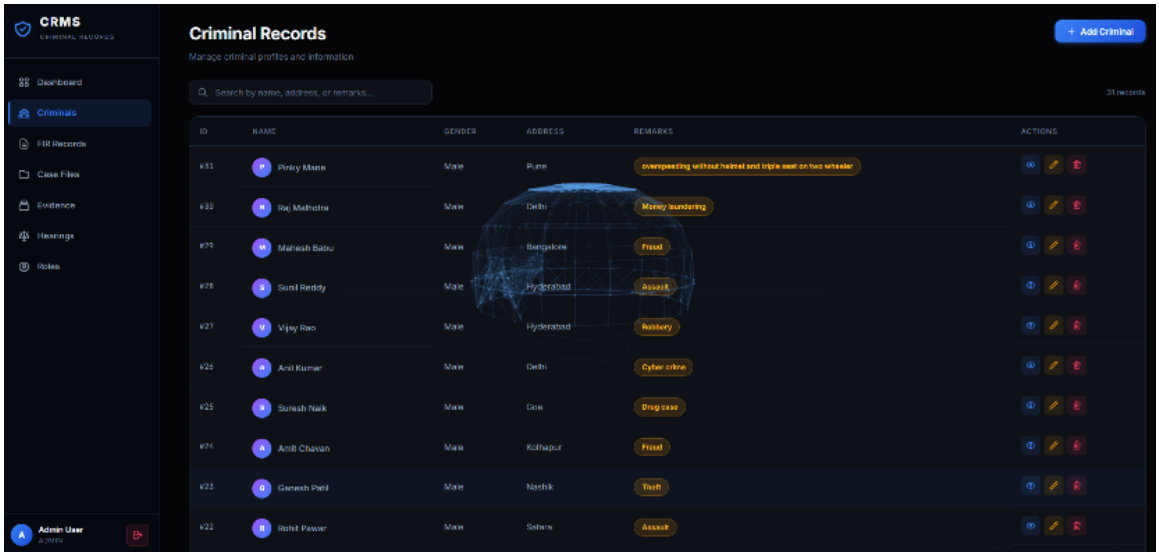
The dashboard acts as the central hub of the system. It displays key statistics such as total criminals, active FIRs, case files, and evidence records. Visual elements like charts and summary cards provide quick insights into case status and crime categories. This allows administrators to monitor system activity efficiently and make informed decisions.

7.4 FIR Records Module



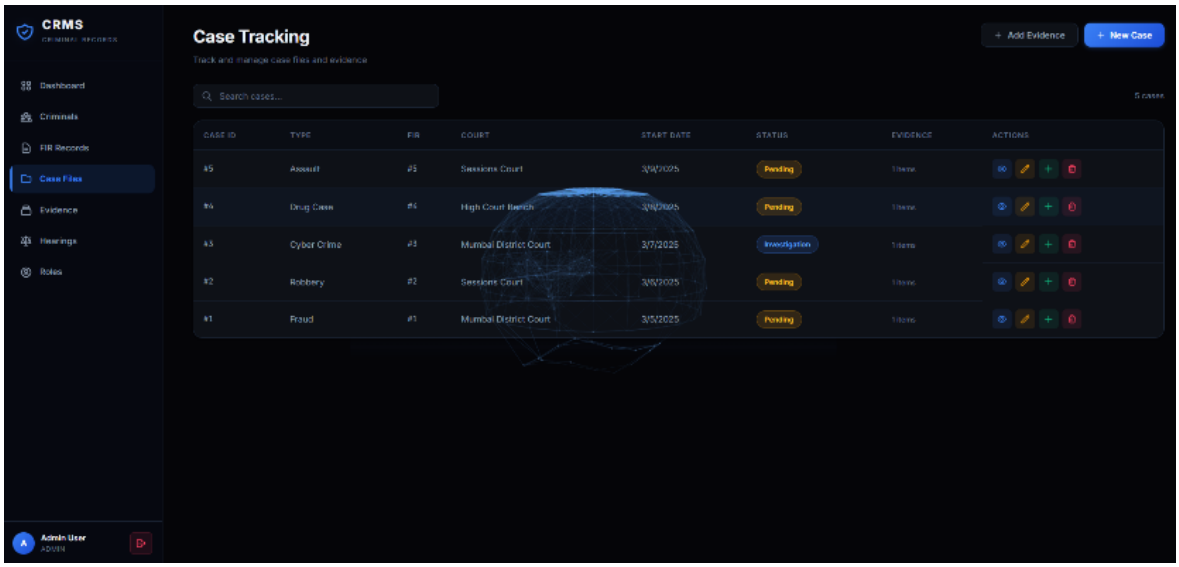
The FIR Records screen displays all registered FIRs in a tabular format. It includes details such as FIR ID, date, time, description, officer assigned, police station, and linked criminals. The module also provides options to add, edit, or delete FIR records, making it easy to manage crime reports systematically.

7.5 Criminal Records Module



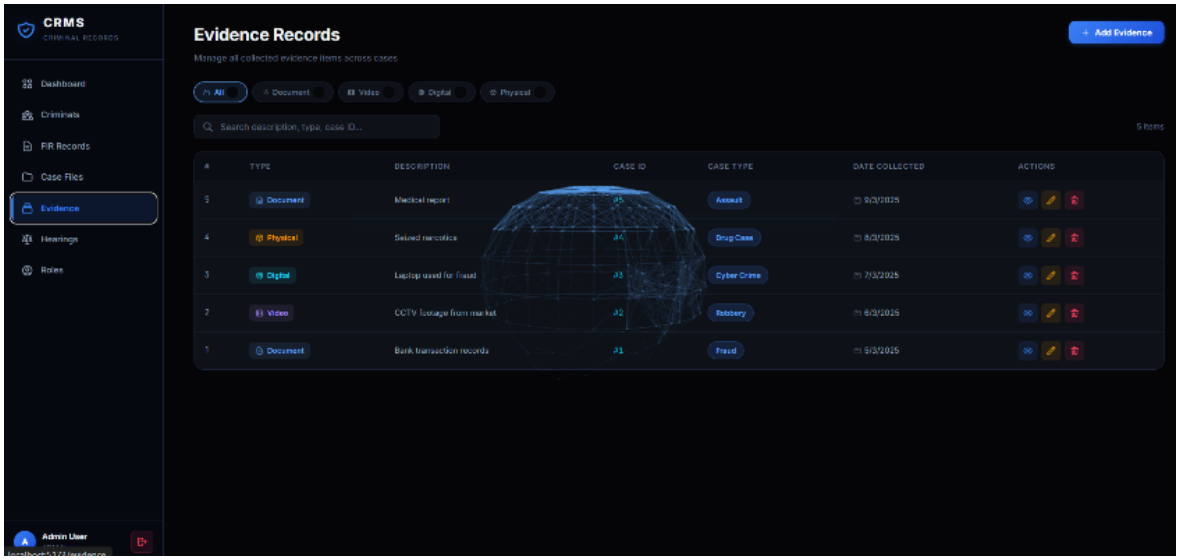
The Criminal Records screen maintains detailed information about individuals involved in crimes. It displays attributes such as criminal ID, name, gender, address, and remarks. The interface allows searching and filtering of records, enabling quick access to specific criminal data.

7.6 Case Tracking Module



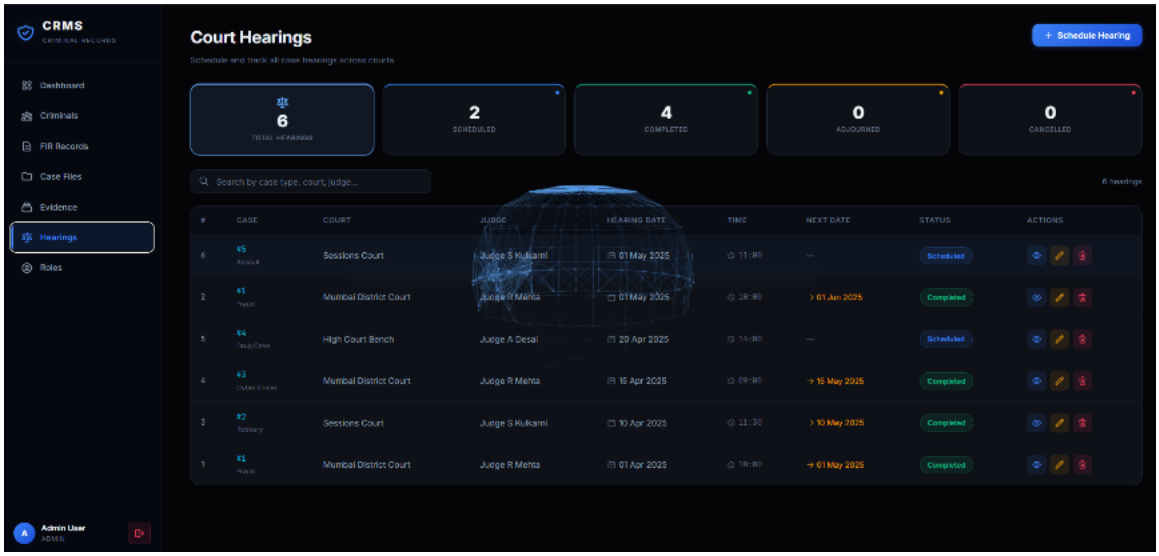
The Case Tracking screen manages all case files linked with FIRs and courts. It includes information such as case ID, case type, FIR reference, court details, start date, and case status. This module helps in tracking the progress of each case from registration to completion.

7.7 Evidence Management



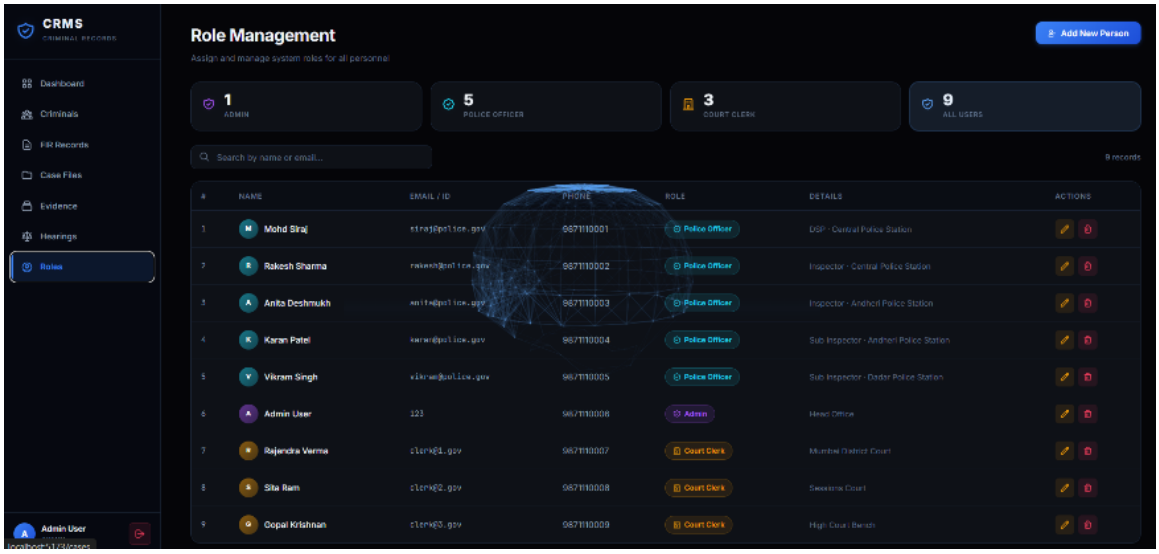
Module The Evidence Records screen stores and manages all evidence collected for different cases. It includes attributes such as evidence type, description, case ID, and date collected. This ensures proper documentation and easy retrieval of evidence for legal proceedings.

7.8 Court Hearings Module



The Court Hearings screen displays scheduled hearings along with case details, court name, judge name, hearing date, time, and status. It helps users track upcoming hearings and monitor completed or adjourned sessions.

7.9 Role Management



Module The Role Management screen allows administrators to manage different users such as police officers, admins, and court clerks. It includes details like name, email, phone number, and assigned role. This module ensures proper access control and system security.

7.10 Screenshots of Backend and Database

Database Tables Overview

The database of the Criminal Record Management System consists of multiple interconnected tables designed to manage law enforcement data efficiently. The following tables are implemented:

Users
Admin
Police_Station
Police_Officer
FIR
Criminal
FIR_Criminal
Criminal_Record
Court
Case_File
Evidence

These tables are related using primary keys and foreign keys to maintain data integrity and support efficient querying

7.10.1 Users and Admin Tables

The Users table stores general user information such as name, email, phone number, and password. The Admin table extends the Users table by storing administrative details.

user_id	name	email	phone	password
1	Mohd Siraj	siraj@police.gov	9871110001	pass
2	Rakesh Sharma	rakesh@police.gov	9871110002	pass
3	Anita Deshmukh	anita@police.gov	9871110003	pass
4	Karan Patel	karan@police.gov	9871110004	pass
5	Vikram Singh	vikram@police.gov	9871110005	pass
6	Admin User	admin@system.com	9871110006	adminpass
NULL	NULL	NULL	NULL	NULL

The Admin table is linked to the Users table through a foreign key, ensuring that every admin is a valid system user.

admin_id	user_id	office	contact
1	6	Head Office	9871110006
NULL	NULL	NULL	NULL

7.10.2 Police_Station and Police_Officer Tables

The Police_Station table stores details of police stations including name, location, and contact number.

station_id	name	location	contact_no
1	Central Police Station	Mumbai	9000011111
2	Andheri Police Station	Mumbai	9000011112
3	Dadar Police Station	Mumbai	9000011113
NULL	NULL	NULL	NULL

The Police_Officer table stores officer-specific details such as badge number, rank, and assigned station. It is linked to both the Users table and Police_Station table using foreign keys, ensuring proper mapping of officers to their stations.

officer_id	user_id	badge_no	rank_1	station_id
101	1	DSP101	DSP	1
102	2	INS102	Inspector	1
103	3	INS103	Inspector	2
104	4	SI104	Sub Inspector	2
105	5	SI105	Sub Inspector	3
NULL	NULL	NULL	NULL	NULL

7.10.3 FIR Table

The FIR (First Information Report) table stores all crime report details including date, time, description, and the officer responsible for the report.

fir_id	date	time	description	officer_id	station_id
1	2025-03-01	10:30:00	Updated Case	102	1
2	2025-03-02	12:10:00	Table Lock Update	103	2
3	2025-03-03	14:45:00	T1 Update	101	1
4	2025-03-05	09:20:00	Drug trafficking report	105	3
5	2025-03-06	16:30:00	Street assault	104	2
11	2025-04-10	11:00:00	Cyber Crime	102	2
NULL	NULL	NULL	NULL	NULL	NULL

It is linked to Police_Officer and Police_Station tables, ensuring that every FIR is associated with a valid officer and station.

7.10.4 Criminal and FIR_Criminal Tables

The Criminal table stores detailed information about individuals involved in crimes, including name, gender, address, and remarks.

criminal_id	name	gender	address	remarks
1	Prithviraj Hiray	Male	Pune	Fraud suspect
2	Aditya Gholasgaon	Male	Aurangabad	Robbery suspect
3	Rahul Patil	Male	Nashik	Theft
4	Aman Khan	Male	Mumbai	Cyber crime
5	Rohan Gupta	Male	Delhi	Fraud
6	Vikas Yadav	Male	Lucknow	Assault
7	Arjun Singh	Male	Jaipur	Robbery
8	Sahil Verma	Male	Chandigarh	Theft
9	Manish Sharma	Male	Delhi	Fraud
10	Rakesh Patel	Male	Ahmedabad	Assault
11	Deepak Jadhav	Male	Pune	Drug case
12	Sanjay More	Male	Kolhapur	Theft
13	Akash Thakur	Male	Nagpur	Cyber crime
14	Vivek Mishra	Male	Varanasi	Fraud
15	Kunal Shah	Male	Surat	Money laundering
16	Rajat Kapoor	Male	Delhi	Assault
17	Sameer Sheikh	Male	Mumbai	Robbery
18	Imran Shaikh	Male	Mumbai	Drug trafficking
19	Nitin Joshi	Male	Pune	Fraud
20	Harsh Mehta	Male	Ahmedabad	Cyber crime
21	Aditya Kulkarni	Male	Pune	Robbery
22	Rohit Pawar	Male	Satara	Assault
23	Ganesh Patil	Male	Nashik	Theft
24	Amit Chavan	Male	Kolhapur	Fraud
25	Suresh Naik	Male	Goa	Drug case
26	Anil Kumar	Male	Delhi	Cyber crime
27	Vijay Rao	Male	Hyderabad	Robbery
28	Sunil Reddy	Male	Hyderabad	Assault
29	Mahesh Babu	Male	Bangalore	Fraud
30	Raj Malhotra	Male	Delhi	Money laundering
31	Pinky Mane	Male	Pune	overspeeding...
NULL	NULL	NULL	NULL	NULL

The FIR_Criminal table is a junction table used to establish a many-to-many relationship between FIR and Criminal tables. This allows multiple criminals to be associated with a single FIR and vice versa.

fir_id	criminal_id
1	1
2	2
3	4
1	5
2	7
4	18
3	20
5	22
NULL	NULL

7.10.5 Criminal_Record Table

The Criminal_Record table stores historical data related to criminals, such as previous offences and remarks.

It is linked to the Criminal table, allowing the system to maintain a detailed criminal history for each individual.

record_id	criminal_id	previous_offence	remarks
1	1	Fraud case 2021	Under investigation
2	2	Robbery case 2022	Repeat offender
4	4	Online scam	Under investigation
5	5	Bank fraud	Repeat offender
NULL	NULL	NULL	NULL

7.10.6 Court Table

The Court table stores information about courts handling cases, including court name, judge name, and location.

This table is used to link legal proceedings with the corresponding case files.

court_id	court_name	judge_name	location
1	Mumbai District Court	Judge R Mehta	Mumbai
2	Sessions Court	Judge S Kulkarni	Mumbai
3	High Court Bench	Judge A Desai	Mumbai
NULL	NULL	NULL	NULL

7.10.7 Case_File Table

The Case_File table manages case-related data, including case ID, FIR reference, court assignment, case type, start date, end date, and status.

It is linked to both FIR and Court tables. A constraint is applied on the status field to ensure only valid values such as “Pending”, “Solved”, and “Investigation” are allowed.

case_id	fir_id	court_id	case_type	start_date	end_date	status
1	1	1	Fraud	2025-03-05	NULL	Solved
2	2	2	Robbery	2025-03-06	NULL	Solved
3	3	1	Cyber Crime	2025-03-07	NULL	Investigation
4	4	3	Drug Case	2025-03-08	NULL	Pending
5	5	2	Assault	2025-03-09	NULL	Pending
11	11	1	Cyber	2025-04-10	NULL	Investigation
NULL	NULL	NULL	NULL	NULL	NULL	NULL

7.10.8 Evidence Table

The Evidence table stores all evidence related to cases, including description, type (Document, Video, Digital, Physical), and date collected.

It is linked to the Case_File table, ensuring that every piece of evidence is associated with a valid case.

evidence_id	case_id	description	type	date_collect...
1	1	Bank transaction records	Document	2025-03-05
2	2	CCTV footage from market	Video	2025-03-06
3	3	Laptop used for fraud	Digital	2025-03-07
4	4	Seized narcotics	Physical	2025-03-08
5	5	Medical report	Document	2025-03-09
11	11	Laptop Logs	Digital	2025-04-10
NULL	NULL	NULL	NULL	NULL

7.10.9 Transaction and Concurrency Features

The database implementation also includes transaction control mechanisms such as START TRANSACTION, COMMIT, ROLLBACK, and SAVEPOINT.

These are used to ensure data consistency and reliability during operations like inserting FIR records, updating case status, and handling concurrent updates.

Row-level locking using FOR UPDATE and table-level locking mechanisms are also implemented to prevent conflicts during simultaneous transactions.

7.11 Conclusion

The system interface of the Crime Report Management System is designed to be intuitive, efficient, and user-friendly. Each module provides clear and structured output, enabling users to perform operations easily. The integration of all modules ensures smooth workflow and effective management of crime-related data.

REFRENCES

[1] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed., New York, USA: McGraw-Hill, 2020.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Boston, USA: Pearson, 2016.

[3] C. J. Date, *An Introduction to Database Systems*, 8th ed., Boston, USA: Pearson, 2004.

[4] Oracle Corporation, "MySQL Documentation," [Online]. Available: https://dev.mysql.com/doc/

[5] Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/

[6] Express.js, "Express Web Framework Documentation," [Online]. Available: https://expressjs.com/

[7] React Documentation, "React – A JavaScript Library for Building User Interfaces," [Online]. Available: https://react.dev/

[8] Mozilla Developer Network (MDN), "Web Development Documentation," [Online]. Available: https://developer.mozilla.org/

[9] GitHub, "Criminal Record Management System Project Repository," [Online]. Available: https://github.com/Shantanu0913/CRIMINAL-RECORD-SYSTEM

[10] W3Schools, "SQL Tutorial," [Online]. Available: https://www.w3schools.com/sql/